LogiBridge Phase 1 Report

FreightBridge Logistics Pvt. Ltd. | LogiEdge Edge AI pilot

Hardware choice and the sensor pipeline foundation

Contents

Section 1	Task B1: Constraint Triangle and hardware choice	page 2
Section 2	Task B2: Arithmetic intensity and Roofline analysis	page 7
Section 3	Task C1: Sensor simulator	page 10

Each section is a self-contained report with its own calculations, assumptions and open items.

Task B1 — Constraint Triangle Application

LogiEdge: Edge AI cold-chain monitoring | FreightBridge Logistics Pvt. Ltd. | Component B, Module 2

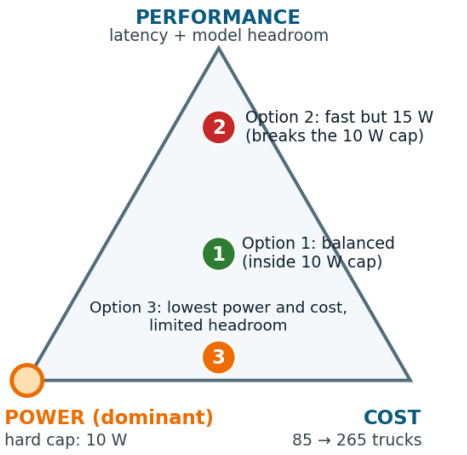
At a glance

Question	Answer
Dominant constraint vertex	Power: the 10 W AI budget on the 12 V truck supply is a hard physical cap
Recommended option	Option 1: Raspberry Pi 5 (8 GB) + AI HAT+ (13 TOPS)
Rejected options	Option 2 breaks the 10 W cap and costs 3×. Option 3 has no headroom and needs custom engineering.
Hardware cost (Option 1)	₹12.75 lakh for 85 trucks; ₹39.75 lakh for 265 trucks

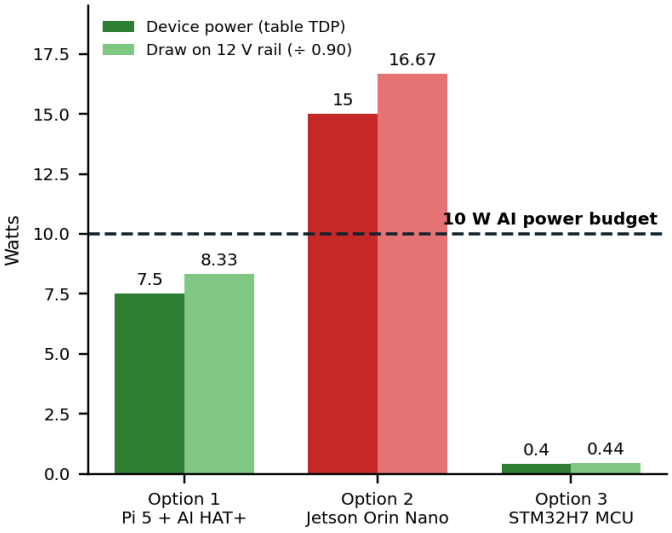
1 The constraint triangle for LogiEdge

Vertex	What it means here	Requirement	Metric used
Performance	Meets the detection deadline and leaves model headroom	Alert within 90 s of a fault	Detection-to-alarm time; compute headroom
Power	Fits the truck's electrical supply	10 W AI budget; 12 V supply via DC-DC converter	Device power and draw on the 12 V rail
Cost	Affordable across the fleet	85 trucks in the pilot; 265 at full scale	Unit price × fleet size

Constraint triangle (schematic placement)



Power against the 10 W budget



2 Applying the triangle to each option

2.1 Performance: the 90 s latency requirement

Option	Inference time (assumed)	Detection-to-alarm	Share of 90 s	Peak compute used
1 Pi 5 + AI HAT+	≈ 0.01 s	10.15 s	11.3%	0.0015% of 13 TOPS
2 Jetson Orin Nano	< 0.01 s	10.15 s	11.3%	0.0003% of 67 TOPS
3 STM32H7 MCU	≈ 0.02 s	10.15 s	11.3%	40% of ≈ 0.5 GOPS

Finding: all three alert in about 10.15 s (10 s window + 0.05 s inference allowance + 0.1 s alarm, as in Task A1), leaving 79.85 s spare. Latency does not separate the options; the window length dominates, not the processor.

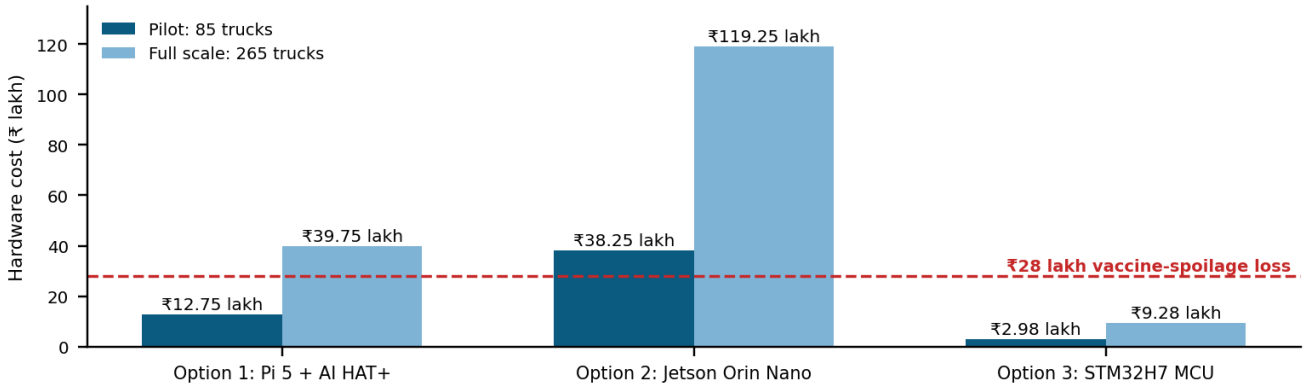
2.2 Power: the 10 W AI budget on a 12 V supply

Option	Device power	Draw on 12 V rail (÷ 0.90)	Current at 12 V	Share of 10 W (device / rail)	Energy per day (24 h, rail)
1 Pi 5 + AI HAT+	7.5 W	8.33 W	0.69 A	75% / 83%	200 Wh
2 Jetson Orin Nano	15 W	16.67 W	1.39 A	150% / 167%	400 Wh
3 STM32H7 MCU	0.4 W	0.44 W	0.04 A	4% / 4%	10.7 Wh

Finding: Option 1 passes whether the 10 W limit applies at the device (2.5 W spare) or on the 12 V rail (1.67 W spare). Option 2 fails under both readings.

2.3 Cost: 85-truck pilot and 265-truck scale-up

Option	Per truck	Pilot (85 trucks)	Full scale (265 trucks)	Relative to Option 1
1 Pi 5 + AI HAT+	₹15,000	₹12,75,000 (₹12.75 lakh)	₹39,75,000 (₹39.75 lakh)	1.0×
2 Jetson Orin Nano	₹45,000	₹38,25,000 (₹38.25 lakh)	₹1,19,25,000 (₹1.19 crore)	3.0×
3 STM32H7 MCU	₹3,500	₹2,97,500 (₹2.98 lakh)	₹9,27,500 (₹9.28 lakh)	0.23×



Prices are the hardware figures given in the brief. They exclude enclosure, DC-DC converter, modem, installation and any custom-board engineering.

3 Dominant constraint vertex: Power

- **Latency does not decide it:** every option alerts in about 10.15 s against a 90 s limit.
- **Power is pass or fail:** Option 2 exceeds the 10 W cap by 50% at the device (15 W) and 67% on the rail (16.67 W). No software setting changes the 15 W figure given for moderate load.
- **Power also drives running cost:** energy per day, heat inside a sealed truck enclosure, and DC-DC converter sizing all scale with it. At 24 h operation Option 1 uses 200 Wh/day against 400 Wh/day for Option 2.
- **Cost breaks the tie:** between the two options that pass the power gate, cost and integration effort separate them.

Conclusion: Power is the dominant vertex. It is the only vertex that removes an option on a physical limit rather than a preference. Cost then separates the survivors, and performance is satisfied by all three.

4 Recommendation: argue for Option 1, against Options 2 and 3

Option	Verdict	Against the 90 s latency requirement	Against the 10 W budget	Against fleet-scale cost
1 Pi 5 + AI HAT+	**Choose**	10.15 s, 11% of the budget; 13 TOPS is vast headroom	7.5 W device (75%); 8.33 W on the rail (83%); passes both readings	₹12.75 lakh pilot, about 46% of one ₹28 lakh spoilage loss; ₹39.75 lakh at 265
2 Jetson Orin Nano	**Reject**	Also meets 90 s, but 67 TOPS is wasted on a model needing about 0.2 GOPS	15 W is 150% of the budget (167% on the rail): fails	₹38.25 lakh pilot, 137% of one ₹28 lakh loss; ₹1.19 crore at 265, 3× Option 1
3 STM32H7 MCU	**Reject (narrowly)**	Meets 90 s: about 0.02 s per inference, so compute is sufficient	Best at 0.4 W (4%)	Cheapest: ₹2.98 lakh pilot, ₹9.28 lakh at 265, before custom-engineering cost

Why Option 1

- It meets the 90 s and 10 W requirements with margin and costs about a third of Option 2 per truck.
- It runs Linux, so the Task A2 design (local MQTT broker, encrypted hash-chained alert log, TLS 1.3 uplink, sync agent, over-the-air updates) uses standard software instead of custom firmware.
- 13 TOPS leaves room for larger models when the system scales to all 265 trucks, including engine-anomaly detection, since the two earlier breakdowns were visible in sensor data three days before.

Why not Option 2

- It fails the hard power cap. NVIDIA lists a 7 W reference mode for the Orin Nano 8 GB (the original modes are 7 W and 15 W), which could fit under 10 W at reduced performance; NVIDIA's table for the Super software lists 15 W, 25 W and MAXN SUPER instead, so check the 7 W mode is available on your kit. Even so, the cost stays at ₹45,000 per truck and the extra 67 TOPS would go unused.
- It is a developer kit, which is not built for vehicle vibration and temperature.

Why not Option 3

- It is technically capable of today's classifier, so the rejection is about integration, not speed. It has about 1 MB of RAM and no Linux, so the local MQTT broker, encrypted log, TLS and updates would need custom firmware, changing the Task A2 design.
- Custom hardware carries one-off engineering cost (board design, firmware, vehicle testing) that is not in the ₹3,500. Option 3 saves ₹9.78 lakh at 85 trucks and ₹30.48 lakh at 265 trucks against Option 1, so it wins on cost only if that engineering stays below those amounts.
- It has little headroom for larger models at scale.

5 Risks of Option 1 and mitigations

Risk	Why it matters	Mitigation
Thin power margin	1.67 W spare on the rail; modem and sensors add load	Measure power under full load with the HAT+ running; confirm whether the modem sits outside the 10 W
Not automotive-rated	Vibration, heat, dust and supply spikes	Industrial enclosure, automotive-grade 12 V DC-DC converter with surge protection, industrial storage instead of an SD card
Accelerator compatibility	Vibration models may use operations the Hailo-8L does not support	Design the model for the accelerator, or run it on the Pi 5 CPU, since the workload is tiny
Parked-truck battery drain	200 Wh/day is about 17% of an assumed 1,200 Wh battery	Low-voltage cut-off and a low-power idle mode when the engine is off

6 Items to confirm

- The three vertices used here are Performance, Power and Cost. If your module defines the triangle differently (for example accuracy, latency and power), tell me and I will remap it.
- Whether the 10 W budget applies at the device or on the 12 V rail (both are shown; Option 1 passes either way).
- DC-DC efficiency (90% assumed) and whether the 4G modem and sensors sit inside the 10 W.
- Model size (5 million multiply-accumulates assumed); a much larger model reduces Option 3's headroom.
- Battery capacity (12 V, 100 Ah assumed) and whether the node stays powered when the engine is off.
- Any quoted engineering cost for the custom MCU board, to test the Option 3 break-even.

Appendix B1 — Assumptions and Workings

Decimal units. “Assumed” values are engineering estimates, not measurements.

A. Assumptions

Item	Value
DC-DC converter efficiency	90% (assumed)
Operating time	24 h per day (assumed)
Model workload	5 million multiply-accumulates = 10 million operations per 10 s window (assumed)
Inference deadline	0.05 s allowance, as in Task A1
Cortex-M7 effective throughput	≈ 0.5 GOPS with optimised int8 kernels (assumed)
Battery	12 V, 100 Ah = 1,200 Wh (assumed)
Prices, device power, TOPS	As given in the brief
Spoilage loss for comparison	₹28 lakh (from the brief)

B. Workings

Step	Calculation	Result
Detection-to-alarm	10 + 0.05 + 0.1	10.15 s; 10.15 ÷ 90 = 11.3%; spare 90 − 10.15 = 79.85 s
Peak compute needed	10 million operations ÷ 0.05 s	0.2 GOPS
Utilisation, Option 1	0.2 GOPS ÷ 13 TOPS	0.0015%
Utilisation, Option 2	0.2 GOPS ÷ 67 TOPS	0.0003%
Option 3 inference time	10 million ÷ 0.5 GOPS; utilisation 0.2 ÷ 0.5	0.02 s; 40%
Rail draw, Option 1	7.5 ÷ 0.90	8.33 W; 0.69 A at 12 V; 200 Wh/day
Rail draw, Option 2	15 ÷ 0.90	16.67 W; 1.39 A at 12 V; 400 Wh/day
Rail draw, Option 3	0.4 ÷ 0.90	0.44 W; 0.04 A at 12 V; 10.7 Wh/day
Share of 10 W, Option 1	7.5 ÷ 10; 8.33 ÷ 10	75%; 83%
Share of 10 W, Option 2	15 ÷ 10; 16.67 ÷ 10	150%; 167%
Battery share per day	200, 400, 10.7 ÷ 1,200 Wh	16.7%; 33.3%; 0.9%
Option 1 cost	15,000 × 85; 15,000 × 265	₹12,75,000; ₹39,75,000
Option 2 cost	45,000 × 85; 45,000 × 265	₹38,25,000; ₹1,19,25,000
Option 3 cost	3,500 × 85; 3,500 × 265	₹2,97,500; ₹9,27,500
Per-truck ratio	45,000 ÷ 15,000; 3,500 ÷ 15,000	3.0×; 0.23×
Pilot cost vs one loss	12.75 ÷ 28; 38.25 ÷ 28	46%; 137%
Option 3 break-even	(15,000 − 3,500) × 85; × 265	₹9,77,500 (₹9.78 lakh); ₹30,47,500 (₹30.48 lakh)

Task B2 — Roofline Analysis

Arithmetic intensity and ridge point | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component B, Module 2

At a glance

Question	Answer
Arithmetic intensity	2.5 FLOP/byte (45 MFLOP ÷ 18 MB)
Ridge point	1.33 FLOP/byte (16 GFLOP/s ÷ 12 GB/s)
Classification	Compute-bound: intensity is 1.875× the ridge point
Latency lower bound	2.81 ms (compute) against 1.5 ms (memory)
Optimisation that helps	Cut FLOPs or raise the compute ceiling: INT8 quantisation, pruning, a lighter architecture

1 Inputs and method

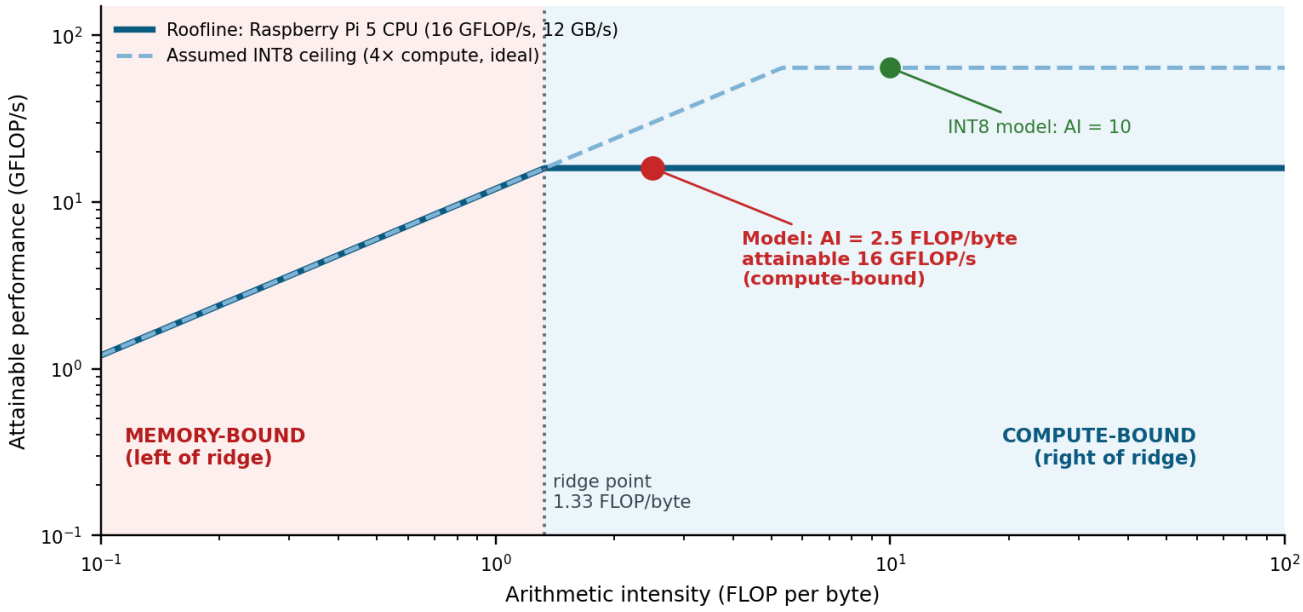
Quantity	Value	Source
Work per inference	≈ 45 MFLOP	Given
Data moved per inference	≈ 18 MB (weights + activations)	Given
Peak compute, Raspberry Pi 5 CPU (NEON)	≈ 16 GFLOP/s	Given
Memory bandwidth, LPDDR4X	≈ 12 GB/s	Given

Roofline relation	Formula
Arithmetic intensity (AI)	FLOPs ÷ bytes moved
Ridge point	Peak compute ÷ memory bandwidth
Attainable performance	min(peak compute, bandwidth × AI)
Latency lower bound	max(FLOPs ÷ peak compute, bytes ÷ bandwidth)

2 Calculation, step by step

Step	Calculation	Result
1 Arithmetic intensity	45×10^6 FLOP ÷ 18×10^6 B	2.5 FLOP/byte
2 Ridge point	16×10^9 FLOP/s ÷ 12×10^9 B/s	1.33 FLOP/byte
3 Compare	2.5 ÷ 1.33	1.875: model sits right of the ridge
4 Bandwidth roof at AI = 2.5	12 GB/s × 2.5	30 GFLOP/s (above the compute ceiling)
5 Attainable performance	min(16, 30)	16 GFLOP/s: the compute ceiling
6 Compute time	$45 \times 10^6 \div 16 \times 10^9$	2.81 ms
7 Memory time	$18 \times 10^6 \div 12 \times 10^9$	1.5 ms
8 Latency lower bound	max(2.81, 1.5)	2.81 ms, set by compute

Classification: the model is compute-bound. Its arithmetic intensity (2.5) is above the ridge point (1.33), so the 12 GB/s memory system could feed 30 GFLOP/s but the CPU can only deliver 16. Compute time (2.81 ms) is 1.875× the memory time (1.5 ms).



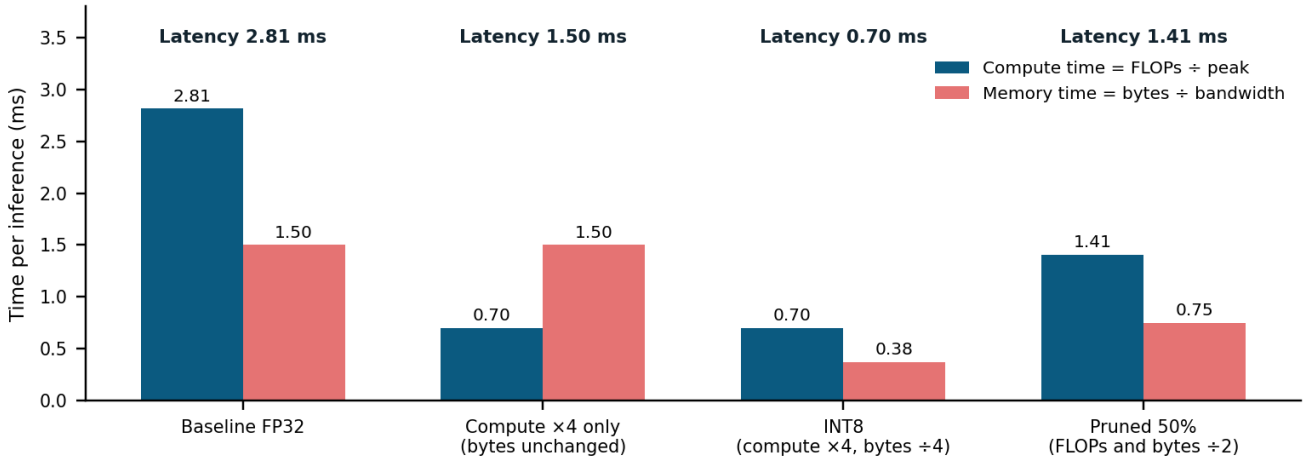
3 Which optimisations the Roofline model favours

Optimisation	Effect on the Roofline	Improves latency?
INT8 quantisation	Cuts bytes about 4× and, with NEON int8 instructions, raises the compute ceiling	**Yes**: attacks both the compute limit and, later, the memory limit
Pruning or a lighter architecture	Cuts FLOPs, so compute time falls in proportion	**Yes**: directly shortens the dominant term
Using all CPU cores, or an NPU such as the Hailo-8L	Raises the compute ceiling	**Yes**, but capped at 1.875× unless bytes also fall (see below)
Operator fusion or tiling to cut memory traffic	Moves the model right, but it is already under the compute roof	**No** gain now: memory time (1.5 ms) is not the bottleneck
Faster or wider RAM	Raises the slanted roof only	**No** gain now: same reason

4 What each optimisation buys: scenario workings

Scenario	AI	Ridge	Compute time	Memory time	Latency	Bound	Gain
Baseline FP32	2.5	1.33	2.81 ms	1.50 ms	2.81 ms	Compute	1.0×
Compute ×4 only (bytes unchanged)	2.5	5.33	0.70 ms	1.50 ms	1.50 ms	Memory	1.875×
INT8 (compute ×4, bytes ÷ 4)	10	5.33	0.70 ms	0.375 ms	0.70 ms	Compute	4.0×
Pruned 50% (FLOPs and bytes ÷ 2)	2.5	1.33	1.41 ms	0.75 ms	1.41 ms	Compute	2.0×

AI = arithmetic intensity in FLOP per byte; ridge = peak compute ÷ bandwidth.



Roofline insight: speeding up compute alone has a ceiling. With bytes unchanged, latency cannot fall below the memory time of 1.5 ms, so the best gain is 2.81 ÷ 1.5 = 1.875× before the model turns memory-bound. INT8 quantisation avoids that ceiling because it shrinks the bytes as well as raising the compute rate.

The 4× INT8 compute gain is an idealised assumption; real NEON gains are usually lower and should be measured. The 50% pruning case assumes bytes fall in proportion to FLOPs.

5 Recommendation and link to earlier tasks

- **Recommended:** INT8 post-training or quantisation-aware training first, then structured pruning if more speed is wanted. Together they cut FLOPs and bytes and keep the model compute-bound at a much shorter latency.
- **Not recommended now:** memory-traffic work (fusion, tiling, faster RAM). The Roofline model shows it would not change latency while the model is compute-bound.
- **Against the 90 s requirement:** the baseline already takes 2.81 ms, which is 0.003% of 90 s and 5.6% of the 0.05 s inference allowance used in Task A1. Optimising is for headroom, battery energy and running more models, not for compliance.
- **Effect on Task B1:** this model (45 MFLOP) is larger than the 10 MFLOP estimate used there. Option 3 (≈ 0.5 GOPS) would take $45 \times 10^6 \div 0.5 \times 10^9 = 0.09$ s, above the 0.05 s allowance but still negligible against 90 s. The Option 1 recommendation stands, and its 2.81 ms CPU time is comfortable.

6 Items to confirm

- Whether 16 GFLOP/s is the single-core or the all-core figure. If it is per core, using four cores could raise the ceiling and the ridge point.
- Whether the 18 MB is FP32 data. The INT8 byte reduction assumes 4 bytes become 1.
- Whether the 45 MFLOP counts a multiply-accumulate as two operations, as assumed here.
- Cache reuse: if part of the 18 MB stays in cache, real traffic is lower, which raises intensity and strengthens the compute-bound result.
- A measured inference time on the Pi 5, to check the 2.81 ms bound.

Task C1 — Sensor Simulator

Python simulator publishing to Mosquitto | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component C, Module 3

At a glance

Item	Detail
Deliverables	`simulator.py` (the simulator) and `test_simulator.py` (23 tests)
Required flag	`--anomaly {none|temp_drift|vibration|combined}`, default none
Broker	Mosquitto on `localhost:1883`, using the paho-mqtt library
Streams	Temperature 1 Hz, vibration RMS 0.5 Hz, door open/close events
Verification	All 23 tests pass; the MQTT path was tested against a paho stub, not a live broker (see Section 6)

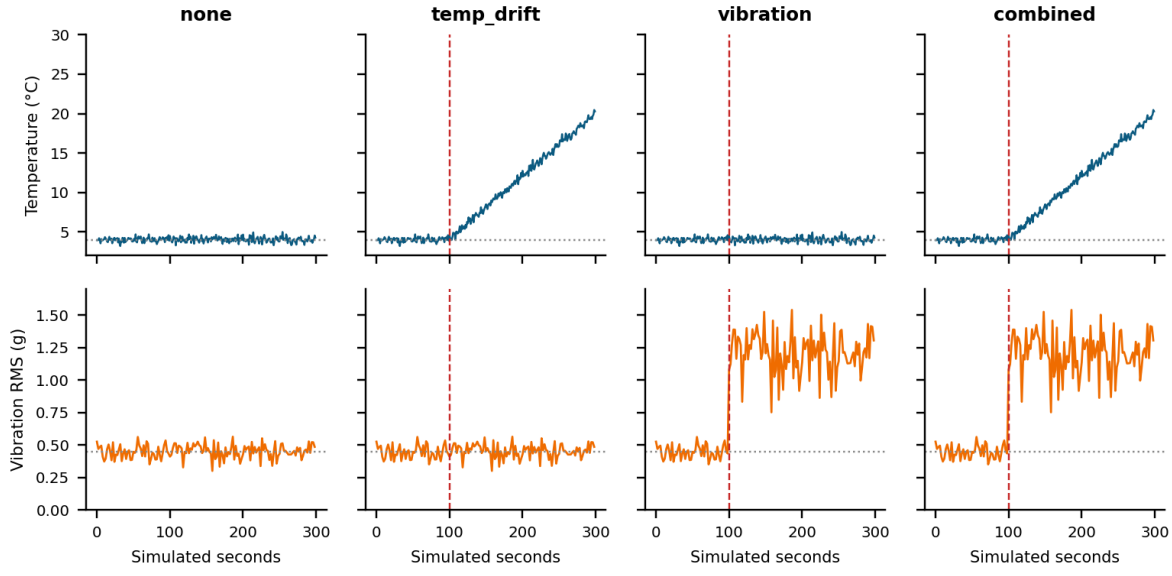
1 Stream design

Stream	Topic	Rate	QoS	Normal	Anomaly
temperature	truck/ID/temp	1 Hz	0	N(4.0 °C, 0.3); setpoint 4 °C	+0.08 °C per reading, linear drift
vibration_rms	truck/ID/vib	0.5 Hz	0	N(0.45 g, 0.05); compressor RMS	Step to N(1.2 g, 0.15); bearing wear
door_event	truck/ID/door	Discrete	1	OPEN or CLOSE with timestamp	Continues in every mode

Topic names and QoS levels follow the Task A2 design. A retained status message (online or offline, with an MQTT last will) is also published on truck/ID/status.

2 Anomaly modes

--anomaly	Temperature	Vibration	Door events
none	Normal	Normal	Normal
temp_drift	Drifts +0.08 °C per reading	Normal	Normal
vibration	Normal	Steps to N(1.2 g, 0.15)	Normal
combined	Drifts +0.08 °C per reading	Steps to N(1.2 g, 0.15)	Normal



Dashed red line: anomaly onset (--anomaly-start 100). Dotted grey line: normal mean (4.0 °C, 0.45 g).

Figure generated from the simulator's own output (seed 42, door events not plotted).

3 Message format

```
truck/T001/temp  qos=0  {"truck_id":"T001","ts":"2026-10-24T11:41:30.520Z","seq":1,"value":4.47,"unit":"C"}
truck/T001/vib   qos=0  {"truck_id":"T001","ts":"2026-10-24T11:41:30.520Z","seq":1,"value":0.567,"unit":"g_rms"}
truck/T001/door  qos=1  {"truck_id":"T001","ts":"2026-10-24T11:41:31.520Z","seq":1,"event":"OPEN"}
```

- Payloads are compact JSON of 83 B (temperature), 88 B (vibration) and 74 B (door event). They stay on the truck, so they do not affect the cellular figures in Task A1.
- Timestamps are UTC with millisecond precision, taken from the scheduled time, so readings are evenly spaced even if the operating system delays a send.
- **seq** counts messages per stream, so a subscriber can detect gaps.

4 Command-line reference

Flag	Default	Purpose
--anomaly	none	none, temp_drift, vibration or combined
--host, --port	localhost, 1883	Mosquitto broker address
--username, --password	none	Only if the broker requires authentication
--truck-id	T001	Identifier used in the topics
--duration	0	Simulated seconds to run; 0 runs until Ctrl-C
--speed	1.0	Simulated seconds per real second, for fast test runs
--seed	random	Makes the data reproducible
--anomaly-start	0	Simulated seconds of normal data before the anomaly begins
--door-interval	300	Mean seconds between door openings
--temp-cap	none	Optional temperature ceiling, e.g. 35 for ambient
--connect-timeout	10	Seconds to wait for the broker before exiting with an error
--dry-run, -v	off	Print messages instead of publishing; log every message

5 How to run it

```
# 1. Install the client library and start the broker
pip install paho-mqtt
mosquitto -v                          # or: sudo systemctl start mosquitto

# 2. In a second terminal, watch every topic for truck T001
mosquitto_sub -h localhost -t 'truck/T001/#' -v

# 3. In a third terminal, run the simulator
python3 simulator.py --anomaly none
python3 simulator.py --anomaly temp_drift --anomaly-start 60 --temp-cap 35
python3 simulator.py --anomaly vibration --seed 1
python3 simulator.py --anomaly combined --duration 600 --speed 10

# 4. Run the tests (no broker needed)
python3 -m unittest -v test_simulator
```

6 Verification

Check	What it confirms	Result
Sensor statistics	Normal temperature and vibration match N(4.0, 0.3) and N(0.45, 0.05); fault vibration matches N(1.2, 0.15)	Pass
Linear drift	Fitted slope is 0.080 °C per reading (tolerance 0.002)	Pass
Rates and spacing	100 temperature and 50 vibration messages per 100 s, spaced exactly 1 s and 2 s	Pass
Anomaly modes	none, temp_drift, vibration and combined each behave as specified; onset delay works	Pass
Door events	Start with OPEN, strictly alternate, open for 20–120 s, continue in every mode	Pass
Reproducibility	Same seed gives identical data; a different seed differs	Pass
CLI	Defaults correct; all four modes accepted; invalid mode rejected with exit code 2	Pass
MQTT adapter (paho stub)	Connects to localhost:1883, sets last will, publishes correct topics, QoS and retain flags, sends online and offline status, disconnects cleanly; exits with code 1 if no broker	Pass
Real-time run	10 s run took 10.09 s and sent 10 temperature and 5 vibration messages	Pass
Ctrl-C	Stops cleanly with exit code 0 and a summary line	Pass
Live Mosquitto	Not run: paho-mqtt and Mosquitto are not available in my environment	Please run Section 5

Observed over 3,600 s (seed 42)	Temperature	Vibration RMS
none	mean 4.00 °C, sd 0.30	mean 0.451 g, sd 0.050
vibration	mean 4.00 °C, sd 0.30	mean 1.202 g, sd 0.150
temp_drift	slope 0.0800 °C per reading, residual sd 0.301	mean 0.451 g, sd 0.050

7 Design notes and items to confirm

- **Drift is fast.** 0.08 °C per reading at 1 Hz is 4.8 °C per minute, 4.8× the 1 °C per minute used in the Task A1 latency analysis. Cargo reaches +1 °C after about 13 s and +3 °C after about 38 s. Without a cap it passes a 35 °C ambient after about 6.5 minutes (31 ÷ 0.08 = 388 readings), so use `--temp-cap 35` for long runs.
- **Vibration is RMS at 0.5 Hz**, one value every 2 s, as in the table. Task A2 described raw 500 Hz blocks on the same topic. I kept the topic name `vib`; tell me if you prefer `vib_rms`.
- **Door events in anomaly modes.** I read the table's last row as: door events continue unchanged in every mode, and “combined” means both anomalies together. Please confirm.
- **Door frequency.** The default mean gap is 300 s so short runs show events. Task A1 assumed 50 events per day, which is about `--door-interval 3400`.
- **Anomaly onset.** It starts at time zero by default; use `--anomaly-start` for a normal period first.
- **Broker details.** Confirm whether your Mosquitto needs a username and password, and which paho-mqtt version you use (1.x and 2.x are both supported).